



2.1. Data Length, Shape, and Size

Previous Section:

 [1.2. Handling Unique and Duplicate Values](#)

Contents:

[1. Length of Data](#)

[1.1. Total Number of Samples](#)

[1.2. Total Number of Columns](#)

[1.3. Counting Unique Values](#)

[1.4. Using `len\(\)` in Calculations](#)

[2. Shape and Size of Data](#)

[2.1. DataFrame Shape](#)

[2.2. DataFrame Size](#)

[2.3. Size of Rows and Columns](#)

[Summary](#)

[References](#)

Before selecting specific rows or columns, it is often useful to understand **how large the dataset is**. Knowing the number of rows, columns, and total values helps you quickly inspect a dataset and verify that it has been loaded correctly.

Throughout this section, we will continue using the same synthetic student dataset introduced earlier (but with no missing values).

```

import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))

```

(this dataset will be used throughout the first part of this course)

In this section, we will explore four simple but commonly used attributes and functions:

Method	Description
<code>len(df)</code>	Returns the number of rows in the DataFrame.
<code>len(df.columns)</code>	Returns the number of columns.
<code>df.shape</code>	Returns a tuple <code>(rows, columns)</code> .
<code>df.size</code>	Returns the total number of elements <code>(rows × columns)</code> .

These methods are frequently used during data exploration and are often the first commands analysts execute after loading a dataset.

1. Length of Data

The built-in `len()` function returns the number of items in an object. When applied to a DataFrame, it returns the **number of rows (samples)**.

1.1. Total Number of Samples

```
print(len(df))
```

Output: 100

Since our dataset contains 100 students, `len(df)` returns **100**.

1.2. Total Number of Columns

To determine how many variables (columns) are in the dataset, apply `len()` to `df.columns`.

```
print(len(df.columns))
```

Output: 4

The dataset contains four columns: Name; Age; Major; GPA

1.3. Counting Unique Values

The `len()` function is also useful after obtaining unique values from a column. For example, to count how many different majors exist:

```
print(df["Major"].unique())  
print(len(df["Major"].unique()))
```

Output:

```
['AI' 'CSC' 'IT' 'SW']
```

4

This technique works for any column and is commonly used to determine the number of distinct categories.

1.4. Using `len()` in Calculations

Because `len()` returns the number of rows, it can also be combined with functions like `sum()` to perform simple calculations.

For example, we can manually compute the average GPA:

```
average_gpa = sum(df["GPA"]) / len(df["GPA"])  
print(average_gpa)
```

You can also use `sum(df["GPA"]) / len(df)`

Output: 3.0231000000000017

Although Pandas already provides `df["GPA"].mean()`, this example demonstrates how `len()` can be combined with other functions.

2. Shape and Size of Data

While `len()` provides only the number of rows, Pandas offers two additional attributes that describe the overall dimensions of a dataset.

2.1. DataFrame Shape

The `shape` attribute returns a tuple containing: `(rows, columns)`

```
print(df.shape)
```

Output: (100, 4)

The first number represents the number of rows, while the second represents the number of columns.

- Individual dimensions can also be accessed using indexing.

```
print("Rows:", df.shape[0])  
print("Columns:", df.shape[1])
```

Output:

Rows: 100

Columns: 4

2.2. DataFrame Size

The `size` attribute returns the total number of elements stored in the DataFrame.

```
print(df.size)
```

Output: 400

Since the dataset contains 100 rows and 4 columns: $100 \times 4 = 400$. Therefore,
`df.size == df.shape[0] * df.shape[1]`

2.3. Size of Rows and Columns

The `size` attribute can also be applied to individual rows or columns because they are represented as Pandas `Series`.

- For example:

```
print(df["GPA"].size)
```

Output: 100

- Similarly, selecting a single row:

```
print(df.iloc[0].size)
```

Output: 4

The first result indicates there are 100 GPA values, while the second shows that each row contains four values.

Summary

This section introduces various ways to retrieve the length, or shape, or size of the data.

- `len(df)` returns the number of rows (samples) in a DataFrame.
- `len(df.columns)` returns the number of columns.
- `len()` can also be used to count unique values and in calculations such as manually computing an average.
- `df.shape` returns a tuple `(rows, columns)`.
- `df.shape[0]` gives the number of rows, while `df.shape[1]` gives the number of columns.
- `df.size` returns the total number of elements (`rows × columns`).
- The `size` attribute also works on individual rows and columns (`Series`).

To practice further, complete Lab 02 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.size.html>

<https://www.geeksforgeeks.org/python/python-pandas-df-size-df-shape-and-df-ndim/>

Next Section:

 [2.2. Data Indexing and Slicing](#)